

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
3
0 5 2
1 3 1
2 2 3
```

Sample Output 1

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67

Average Waiting Time = 3.33

Example 2

Sample Input 2

4

0 4 3

0 3 1

2 2 4

5 1 2

Sample Output 2

Enter number of processes:

Enter AT, BT and Priority for P1:

Enter AT, BT and Priority for P2:

Enter AT, BT and Priority for P3:

Enter AT, BT and Priority for P4:

Average Turnaround Time = 5.25

Average Waiting Time = 2.75

Solution:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    int at[100], bt[100], pr[100];
```

```
int ct[100], tat[100], wt[100], done[100];
```

```
int idx, min_pr;
```

```
float avg_tat = 0, avg_wt = 0;
```

```
printf("Enter number of processes: \n");
```

```
scanf("%d", &n);
```

```
for(i = 0; i < n; i++) {
```

```
    printf("Enter AT, BT and Priority for P%d: \n", i+1);
```

```
    scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
```

```
    done[i] = 0;
```

```
}
```

```
while(completed < n) {
```

```
    idx = -1;
```

```
    min_pr = 9999;
```

```
    for(i = 0; i < n; i++) {
```

```
        if(at[i] <= time && done[i] == 0) {
```

```
            if(pr[i] < min_pr) {
```

```
                min_pr = pr[i];
```

```
                idx = i;
```

```
            }
```

```
        }
```

```
    }
```

```
    if(idx == -1) {
```

```
        time++;
    }
    else {
        time += bt[idx];
        ct[idx] = time;

        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        avg_tat += tat[idx];
        avg_wt += wt[idx];

        done[idx] = 1;
        completed++;
    }
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
printf("Average Waiting Time = %.2f\n", avg_wt);

return 0;
}
```

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where
 - The first integer denotes the Arrival Time (AT)
 - The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

```
3
2
0 5
1 3
2 1
```

Sample Output 1

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
```

Enter AT and BT for P3:

Average Turnaround Time: 6.33

Average Waiting Time: 3.33

Solution:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, tq, i, time = 0, completed = 0;
```

```
    int at[100], bt[100], rt[100];
```

```
    int ct[100], tat[100], wt[100];
```

```
    float avg_tat = 0, avg_wt = 0;
```

```
    printf("Enter number of processes: \n");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Time Quantum: \n");
```

```
    scanf("%d", &tq);
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter AT and BT for P%d: \n", i+1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
        rt[i] = bt[i]; // remaining time
```

```
    }
```

```
    while(completed < n) {
```

```
        int executed = 0;
```

```

for(i = 0; i < n; i++) {
    if(at[i] <= time && rt[i] > 0) {
        executed = 1;

        if(rt[i] > tq) {
            time += tq;
            rt[i] -= tq;
        } else {
            time += rt[i];
            ct[i] = time;
            rt[i] = 0;
            completed++;

            tat[i] = ct[i] - at[i];
            wt[i] = tat[i] - bt[i];

            avg_tat += tat[i];
            avg_wt += wt[i];
        }
    }
}

if(executed == 0)
    time++;
}

avg_tat /= n;
avg_wt /= n;

```

```
printf("\nAverage Turnaround Time: %.2f\n", avg_tat);  
printf("Average Waiting Time: %.2f\n", avg_wt);  
  
return 0;  
}
```